

E TRACKER – HOME BASED EXPENSE TRACKING MECHANISM

A project report submitted to ICT Academy of Kerala

in partial fulfillment of the requirements

for the certification of

ADVANCED PYTHON

submitted by

AJU MATHEW THOMSON

GOPIKA S S

K PRANAV ESWAR

SAN JOBIN S

VISHNU S



ICT ACADEMY OF KERALA

THIRUVANANTHAPURAM, KERALA, INDIA

MAY 2026

LIST OF FIGURES

SL NO.	FIGURES	PAGE NO
<i>fig 4.1.1</i>	<i>System architecture</i>	<i>17</i>
<i>fig 11.2.1</i>	<i>Home</i>	<i>35</i>
<i>fig 11.2.2</i>	<i>About</i>	<i>35</i>
<i>fig 11.2.3</i>	<i>Features</i>	<i>36</i>
<i>fig 11.2.4</i>	<i>Contact</i>	<i>36</i>
<i>fig 11.2.5</i>	<i>Login page</i>	<i>37</i>
<i>fig 11.2.6</i>	<i>Signup page</i>	<i>37</i>
<i>fig 11.2.7</i>	<i>User dashboard page</i>	<i>38</i>
<i>fig 11.2.8</i>	<i>Add expense bot</i>	<i>38</i>
<i>fig 11.2.9</i>	<i>Statement</i>	<i>39</i>
<i>fig 11.2.10</i>	<i>Budget planner</i>	<i>39</i>
<i>fig 11.2.11</i>	<i>Analytics – pie chart</i>	<i>40</i>
<i>fig 11.2.12</i>	<i>Analytics – bar chart</i>	<i>40</i>
<i>fig 11.2.13</i>	<i>Analytics – line chart</i>	<i>41</i>
<i>fig 11.2.14</i>	<i>Settings – budget fix</i>	<i>41</i>
<i>fig 11.2.15</i>	<i>Settings – add income</i>	<i>42</i>
<i>fig 11.2.16</i>	<i>Settings – preferences</i>	<i>42</i>
<i>fig 11.2.17</i>	<i>Settings – security</i>	<i>43</i>
<i>fig 11.2.18</i>	<i>Admin dashboard</i>	<i>43</i>
<i>fig 11.2.19</i>	<i>Admin – handling user page</i>	<i>44</i>
<i>fig 11.2.20</i>	<i>Admin – viewing users profile</i>	<i>44</i>
<i>fig 11.2.21</i>	<i>Admin – add / delete categories</i>	<i>45</i>
<i>fig 11.2.22</i>	<i>Admin - settings</i>	<i>45</i>

TABLE OF CONTENTS

<u>CONTENT</u>	<u>PAGE No.</u>
ABSTRACT	4
1. PROBLEM DEFINITION	5
1.1 OVERVIEW	5
1.2 PROBLEM STATEMENT	5
2. INTRODUCTION	8
3. SYSTEM ANALYSIS	8
3.1 EXISTING SYSTEM	8
3.2 PROPOSED SYSTEM	8
3.3 FEASIBILITY STUDY	9
3.4 TECHNOLOGIES USED	11
3.5 LANGUAGE SPECIFICATIONS	12
4. SYSTEM DESIGN	16
4.1 SYSTEM ARCHITECTURE	17
5. PROJECT DESCRIPTION	18
6. SYSTEM TESTING AND IMPLEMENTATION	20
6.1 SYSTEM TESTING	20
6.2 SYSTEM IMPLEMENTATION	21
7. SYSTEM MAINTENANCE	22
8. FUTURE ENHANCEMENTS	24
9. CONCLUSION	25
10. BIBLIOGRAPHY	26
11. APPENDIX	28
11.1 SYSTEM CODING	28
11.2 SCREENSHOTS	35

ABSTRACT

The **ETRACKER – Home Based Budget Tracking Mechanism** is a web-based financial management application developed using the Flask framework. The primary objective of the system is to help individuals and families efficiently manage their day-to-day financial activities in a simple, secure, and organized manner. In modern society, managing household expenses has become increasingly challenging due to rising living costs and the lack of proper financial planning tools. Many people still depend on manual methods such as notebooks or spreadsheets to track expenses, which are often time-consuming, error-prone, and difficult to maintain. The ETRACKER system is designed to overcome these limitations by providing a centralized digital platform for budget management.

The application allows users to securely register and log into the system using authenticated credentials. Once logged in, users can add, update, and manage their income and expense records. Each transaction can be categorized based on expense type, enabling users to better organize and monitor their spending activities. The system also provides features for viewing transaction history, expense summaries, and category-wise analysis, helping users gain a clear understanding of their financial status.

To improve data analysis and visualization, the project incorporates Python libraries such as NumPy, Pandas, and Matplotlib. These libraries are used for efficient data handling, processing, and graphical representation of expense statistics. Through charts and summaries, users can easily identify spending patterns, compare expenses across categories, and make informed financial decisions.

The application is implemented using various functional modules including the **User Authentication Module, Expense Management Module, Dashboard Module, and Admin Module**. The frontend of the system is developed using HTML, CSS, and JavaScript to provide a responsive and user-friendly interface, while the backend is powered by Flask and connected to a database system such as SQLite for secure data storage and retrieval.

Security is an important aspect of the application. The system ensures that each user's financial data remains private through secure login authentication and session management. By automating the process of expense tracking and budget monitoring, ETRACKER reduces manual effort, minimizes calculation errors, and improves overall financial management efficiency.

In conclusion, the ETRACKER system serves as a reliable and effective solution for home-based budget tracking and expense management. It not only simplifies financial record keeping but also encourages users to maintain better financial discipline, improve savings habits, and plan their future expenses more effectively.

1. PROBLEM DEFINITION

1.1 OVERVIEW

Managing personal finances is an essential part of daily life. Many individuals struggle to maintain proper records of their income and expenses due to the lack of an efficient and organized system. Traditional methods such as writing expenses in notebooks or maintaining spreadsheets are often time-consuming, difficult to manage, and prone to calculation errors.

With the increasing use of digital technologies, there is a need for a simple and accessible solution that helps users monitor their financial activities effectively. The **ETRACKER – Home Based Budget Tracking Mechanism** is developed to provide a web-based platform where users can securely manage their expenses, analyze spending patterns, and maintain better control over their budget.

The application is designed using the Flask framework along with modern web technologies to ensure ease of use, reliability, and efficiency.

1.2 PROBLEM STATEMENT

Many people face difficulties in tracking and managing their daily financial transactions efficiently. Existing manual methods lack automation, proper organization, and analytical capabilities, making it difficult for users to monitor their spending habits and maintain financial discipline.

The major problems identified are:

- Difficulty in maintaining accurate expense records
- Time-consuming manual calculations
- Lack of graphical analysis and reports
- Poor organization of financial data
- Risk of data loss and human errors
- Limited accessibility and monitoring features

To overcome these challenges, the **ETRACKER** system is proposed as a web-based budget tracking application that provides secure authentication, expense management, data analysis, and graphical visualization for effective home-based financial management.

2. INTRODUCTION

In the modern era, effective financial management has become an essential part of maintaining a stable and well-organized lifestyle. With increasing daily expenses and changing economic conditions, individuals and families often face difficulties in managing their income and expenditures properly. Many people still depend on traditional methods such as handwritten records, notebooks, or spreadsheets to maintain their financial information. Although these methods may appear simple, they are often time-consuming, difficult to organize, and highly prone to human errors. In addition, manual systems lack proper analytical capabilities, making it difficult for users to identify spending patterns, monitor savings, or make informed financial decisions.

As technology continues to advance, there is a growing need for digital solutions that simplify and improve personal financial management. The ETRACKER – Home Based Budget Tracking Mechanism is developed as a web-based application to address these challenges and provide an efficient solution for managing household budgets and daily expenses. The application is built using the Flask framework, which provides a lightweight and flexible environment for developing secure and scalable web applications.

The main objective of the ETRACKER system is to help users efficiently record, organize, and monitor their financial transactions in a structured manner. The application allows users to add income and expense details, categorize transactions based on their type, and maintain a clear record of their spending activities. By storing financial data digitally, the system eliminates the difficulties associated with manual calculations and record keeping.

One of the major advantages of the application is its simple and user-friendly interface. The system is designed in such a way that even users with minimal technical knowledge can easily navigate and use the application without difficulty. Features such as secure user registration and login ensure that each user's financial information remains private and protected. The system also includes functionalities for updating and deleting transactions, viewing expense history, and accessing summarized financial reports.

To enhance financial analysis and visualization, the project integrates Python libraries such as Pandas, NumPy, and Matplotlib. These tools are used for efficient data processing, statistical analysis, and graphical representation of expense data. Through charts and visual reports, users

can easily understand their spending behavior, identify unnecessary expenses, and plan their future budgets more effectively.

The application consists of multiple modules including the User Authentication Module, Expense Management Module, Dashboard and Visualization Module, and Admin Module. Each module is designed to perform specific tasks efficiently while maintaining smooth interaction between the frontend, backend, and database components. Technologies such as HTML, CSS, and JavaScript are used for frontend development, while Flask handles the backend processing and database communication.

By automating the process of expense tracking and budget management, ETRACKER reduces manual effort, minimizes errors, and improves the overall efficiency of personal financial management. The system encourages users to maintain financial discipline, improve savings habits, and make better financial decisions based on accurate data and analysis.

3. SYSTEM ANALYSIS

System analysis is the process of studying the current system, identifying its limitations, and proposing an improved solution. In the context of the ETRACKER – Home Based Budget Tracking Mechanism, this phase focuses on understanding the drawbacks of existing expense tracking methods and defining the requirements of the proposed system.

3.1 EXISTING SYSTEM

In the existing system, individuals generally manage their finances using manual methods such as notebooks, spreadsheets, or basic mobile applications. These methods have several limitations:

- Lack of proper organization and structure
- High chances of human error during calculations
- No real-time tracking or updates
- Difficulty in analyzing spending patterns
- Absence of graphical reports and summaries
- Limited data security and privacy

Due to these issues, users find it difficult to maintain accurate financial records and make informed decisions.

3.2 PROPOSED SYSTEM

The proposed system, **ETRACKER**, is a web-based application developed using the Flask framework that provides an efficient and secure solution for managing personal finances. The system is designed to simplify expense tracking, improve financial analysis, and provide users with an organized platform for budget management.

FEATURES OF THE PROPOSED SYSTEM

- a) **Secure User Registration and Login System** : The system provides secure authentication features that allow users to register and log in safely. This ensures that personal financial information remains private and protected.
- b) **Easy Recording of Income and Expenses** : Users can easily add, update, and manage their income and expense details through a simple and organized interface.
- c) **Categorization of Transactions** : The application allows transactions to be categorized into sections such as food, travel, shopping, bills, and savings for better organization and tracking.
- d) **Real-Time Tracking of Financial Data** : The system enables users to monitor their financial activities instantly, helping them maintain accurate and up-to-date records.

- e) **Graphical Visualization :** Graphical reports and charts are generated using Matplotlib to help users analyze spending patterns and financial trends effectively.
- f) **Data Processing and Analysis :** The application uses Pandas for efficient data manipulation, analysis, and generation of expense summaries and reports.
- g) **User-Friendly Interface :** The system is designed with a clean and responsive interface that allows users to navigate and operate the application easily, even with basic technical knowledge.

3.3 FEASIBILITY STUDY

3.3.1 TECHNICAL FEASIBILITY

Technical feasibility refers to the process of evaluating whether the proposed system can be developed and implemented using the available technologies, tools, hardware, and technical resources. It determines whether the required technology is practical, reliable, and capable of supporting the successful development and operation of the system.

For the development, the project is technically feasible because it uses modern and widely accepted web technologies that are efficient, reliable, and easy to implement. The application is developed using the Flask framework, which is a lightweight and flexible Python framework suitable for web application development. Flask provides features such as routing, session handling, database connectivity, and secure backend processing, making it appropriate for implementing expense tracking functionalities.

The frontend of the system is designed using HTML, CSS, and JavaScript, which are standard technologies used in web development. HTML is used for structuring the web pages, CSS is used for styling and improving the visual appearance of the application, and JavaScript is used to add interactive features and dynamic behavior to the user interface.

The project also uses SQLite as the database management system for storing user details and expense records securely. SQLite is lightweight, easy to configure, and suitable for small to medium-scale applications.

3.3.2 ECONOMIC FEASIBILITY

Economic feasibility refers to analyzing whether a project can be developed and implemented within a reasonable budget.

The system is economically feasible because it is developed using open-source and freely available technologies such as Flask, Python, HTML, CSS, and JavaScript. The project also uses free Python libraries like NumPy, Pandas, and Matplotlib for data analysis and visualization.

The database used, SQLite, is lightweight and does not require any licensing cost. In addition, the system can run on standard computers without expensive hardware or infrastructure. Since the project reduces manual work and improves financial management efficiency, it saves both time and operational cost. Therefore, the system is considered economically feasible.

3.3.3 OPERATIONAL FEASIBILITY

Operational feasibility refers to evaluating whether the proposed system can function effectively in a real-world environment and whether users can easily operate it.

Our system is operationally feasible because the system is simple, user-friendly, and easy to use. Users can efficiently record and manage their income and expenses through a clear and organized interface. Even individuals with basic computer knowledge can operate the application without difficulty.

The system provides secure login features, expense summaries, and graphical reports, making financial management easier and more effective. Since the application is web-based, it can be accessed through a standard web browser without requiring complex installation or technical expertise. Therefore, the system can be easily adopted and used for daily budget management.

3.4 TECHNOLOGIES USED

Category	Technology / Tool	Purpose
Frontend	HTML	Used to create the structure of web pages
Frontend	CSS	Used for styling and designing the user interface
Frontend	JavaScript	Used to add interactivity and dynamic functionality
Backend	Flask	Used for backend development, routing, and server-side processing
Programming Language	Python	Used for implementing application logic and backend operations
Database	SQLite	Used for storing user and expense data securely
Python Library	NumPy	Used for numerical operations and calculations
Python Library	Pandas	Used for data manipulation and analysis
Python Library	Matplotlib	Used for graphical visualization and charts
Development Tool	Visual Studio Code	Used for writing and managing project code
Version Control	Git & GitHub	Used for version control and project management
Browser	Google Chrome	Used for testing and running the application

3.5 LANGUAGE SPECIFICATIONS

3.5.1 PYTHON

Python is a high-level, interpreted programming language known for its simplicity, readability, and versatility. It was created by Guido van Rossum and was first introduced in 1991. Python was developed with the objective of providing an easy-to-learn and efficient programming language that supports clear and structured coding practices. One of the major benefits of Python is its simple syntax, which allows developers to write programs with fewer lines of code compared to many other programming languages. It also supports multiple programming paradigms such as object-oriented, procedural, and functional programming. Python is widely used in real-life applications including web development, data analysis, artificial intelligence, machine learning, automation, scientific computing, cybersecurity, game development, and software testing. In the ETRACKER project, Python is used as the core programming language for implementing backend logic, handling data processing, and integrating libraries such as NumPy, Pandas, and Matplotlib for analysis and visualization of financial data.

3.5.2 FLASK

Flask is a lightweight and open-source web framework developed in Python for building web applications. It was created by Armin Ronacher and was first released in 2010. Flask is designed to provide developers with a simple and flexible framework for developing secure and scalable web applications. It follows the WSGI (Web Server Gateway Interface) standard and is based on the Werkzeug toolkit and Jinja2 template engine. One of the major advantages of Flask is its simplicity and minimalistic design, which allows developers to build applications quickly with better control over components and functionalities. Flask supports routing, session management, template rendering, and database integration, making it suitable for both small and large-scale applications. Flask is widely used in real-world applications such as web portals, dashboards, content management systems, REST APIs, machine learning applications, and data-driven web platforms. In the ETRACKER project, Flask is used as the backend framework to manage routing, user authentication, session handling, database communication, and overall server-side operations of the application.

3.5.3 HTML

HTML (HyperText Markup Language) is the standard markup language used for creating and structuring web pages. It was developed by Tim Berners-Lee in 1991 to enable the creation and sharing of documents on the World Wide Web. HTML uses tags and elements to define the structure of web content such as headings, paragraphs, forms, tables, images, and links. One of the major advantages of HTML is its simplicity and compatibility with all web browsers. HTML forms the backbone of every website and is widely used in web development, online applications, educational platforms, e-commerce websites, and digital content systems. In the ETRACKER project, HTML is used to design the structure and layout of pages such as login forms, dashboards, expense entry forms, and report sections.

3.5.4 CSS3

CSS3 (Cascading Style Sheets Version 3) is a style sheet language used for designing and enhancing the appearance of web pages. CSS was originally introduced by Håkon Wium Lie in 1996, while CSS3 introduced advanced styling features and modern design capabilities. CSS3 allows developers to apply colors, fonts, layouts, animations, transitions, and responsive designs to web pages. One of the key benefits of CSS3 is that it separates the presentation layer from the content structure, improving maintainability and design flexibility. CSS3 is widely used in websites, mobile-responsive applications, dashboards, and interactive web interfaces. In the ETRACKER project, CSS3 is used to create an attractive and user-friendly interface, improve page layout, and provide responsive design for better user experience.

3.5.5 JavaScript

JavaScript is a high-level scripting language used to add interactivity and dynamic behavior to web applications. It was developed by Brendan Eich in 1995. JavaScript enables web pages to respond to user actions such as button clicks, form validation, animations, and real-time updates. It is one of the core technologies of modern web development alongside HTML and CSS. JavaScript is widely used in interactive websites, web applications, online games, data visualization systems, and real-time communication platforms. One of its major advantages is the ability to create dynamic and responsive user experiences directly within the browser. In the ETRACKER project, JavaScript is used for form validation, interactive dashboard functionality, dynamic content updates, and improving overall user interaction within the application.

3.5.6 SQLite

SQLite is a lightweight, open-source relational database management system used for storing and managing data efficiently. It was developed by D. Richard Hipp in the year 2000. Unlike many other database systems, SQLite does not require a separate server process, making it simple, portable, and easy to use. One of the major advantages of SQLite is its low resource usage and fast performance for small to medium-scale applications. SQLite is widely used in mobile applications, desktop software, embedded systems, and web applications for storing structured data securely. In the ETRACKER project, SQLite is used to store user information, login credentials, income records, and expense transaction details efficiently.

3.5.7 NumPy

NumPy (Numerical Python) is an open-source Python library used for numerical computing and mathematical operations. It was created by Travis Oliphant in 2005. NumPy provides support for multidimensional arrays, matrices, and various mathematical functions, enabling efficient handling of numerical data. One of its major benefits is high-speed computation and efficient memory management when working with large datasets. NumPy is widely used in scientific computing, data analysis, machine learning, engineering applications, and financial systems. In the ETRACKER project, NumPy is used for performing financial calculations and numerical operations related to expense analysis.

3.5.8 Pandas

Pandas is a powerful open-source Python library used for data manipulation, organization, and analysis. It was developed by Wes McKinney in 2008. Pandas provides data structures such as DataFrames and Series, which help in efficiently handling structured data. The library supports operations such as filtering, sorting, grouping, and analyzing datasets. One of the major advantages of Pandas is its ability to process large amounts of data quickly and accurately. Pandas is widely used in data science, business analytics, finance, research, and reporting systems. In the ETRACKER project, Pandas is used to organize and analyze income and expense records for generating summaries and reports.

3.5.9 Matplotlib

Matplotlib is an open-source Python library used for creating graphical visualizations and charts. It was developed by John D. Hunter in 2003. Matplotlib enables developers to generate various types of graphs such as pie charts, bar graphs, line charts, and histograms for data representation and analysis. One of its major benefits is its ability to produce clear and customizable visual outputs that help users understand data more effectively. Matplotlib is widely used in scientific research, financial analysis, machine learning, and reporting applications. In the ETRACKER project, Matplotlib is used to generate graphical reports and visual representations of user expenses and spending patterns.

4. SYSTEM DESIGN

System design is the process of defining the architecture, components, modules, database structure, and overall workflow of a system. It provides a blueprint for the development of the application and explains how different parts of the system interact with each other. The ETRACKER – Home Based Budget Tracking Mechanism is designed as a web-based application that ensures efficient financial management, secure data handling, and user-friendly interaction.

The system follows a modular architecture where each module performs a specific function independently while maintaining smooth communication with other modules. The application is developed using frontend technologies such as HTML, CSS, and JavaScript, while the backend operations are handled using the Flask framework. SQLite is used as the database for storing and managing user and expense-related information securely.

The design of the system focuses on simplicity, efficiency, and scalability. The frontend interface is designed to provide easy navigation and smooth interaction for users. The backend processes user requests, validates data, manages sessions, and communicates with the database. Python libraries such as Pandas, NumPy, and Matplotlib are integrated for data processing, calculations, analysis, and graphical visualization.

The main modules included in the system are:

- User Authentication Module – Handles user registration, login, and session management.
- Expense Management Module – Allows users to add, update, delete, and manage income and expense records.
- Dashboard Module – Displays summaries, total expenses, and graphical reports.
- Visualization Module – Generates charts and financial analysis reports using Matplotlib.
- Database Module – Manages storage and retrieval of user and transaction data.

The system architecture ensures proper communication between the user interface, backend logic, and database system. User requests are processed through Flask routes, validated by the backend, stored in the database, and displayed through interactive web pages.

4.1 SYSTEM ARCHITECTURE

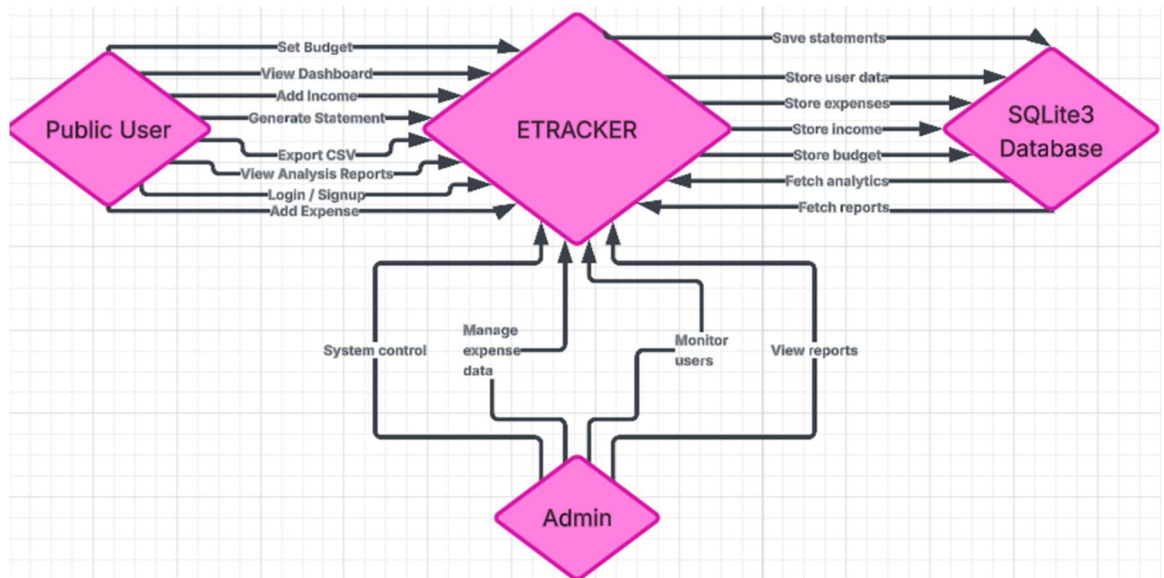


fig 4.1.1

5. PROJECT DESCRIPTION

The ETRACKER – Home Based Budget Tracking Mechanism is a web-based financial management application developed to help individuals efficiently manage their daily income and expenses. The project is designed using the Flask framework along with modern web technologies to provide a secure, user-friendly, and organized platform for personal budget tracking.

The main purpose of the system is to simplify the process of financial management by replacing traditional manual methods with an automated digital solution. Users can securely register and log into the system to maintain their personal financial records. Once authenticated, users can add, update, view, and delete income and expense transactions based on different categories such as food, travel, shopping, bills, and savings.

The application provides a dashboard that displays summarized financial information and graphical reports, helping users analyze their spending habits and monitor their overall financial status. By integrating Python libraries such as Pandas, NumPy, and Matplotlib, the system performs data processing, calculations, and visualization of expense statistics through charts and reports.

The project also utilizes different algorithms and logical techniques for efficient system operation and data processing. Linear search techniques are used for retrieving and filtering transaction records from the database. Sorting operations are applied for organizing expense data based on date, category, or amount. Basic statistical calculations are used for generating expense summaries, total spending, and category-wise analysis. Data visualization algorithms provided through Matplotlib are used to generate pie charts and graphical reports for better financial understanding.

The project is divided into multiple functional modules for better organization and maintainability. The User Authentication Module manages secure login and registration processes. The Expense Management Module handles the recording and management of financial transactions. The Dashboard and Visualization Module generates graphical reports and spending summaries, while the Database Module stores and retrieves user and transaction details securely using SQLite.

The frontend of the application is developed using HTML, CSS, and JavaScript to create a responsive and interactive user interface. The backend is powered by Flask, which manages routing, session handling, data validation, and communication with the database.

The system is designed to be simple and easy to use, allowing even users with basic computer knowledge to operate the application effectively. By automating expense tracking and budget analysis, the ETRACKER system helps users reduce manual effort, avoid financial mismanagement, improve savings habits, and maintain better financial discipline.

6. SYSTEM TESTING AND IMPLEMENTATION

System testing and implementation are important phases in software development that ensure the application functions correctly and meets user requirements. In the ETRACKER – Home Based Budget Tracking Mechanism, testing was performed to verify the accuracy, security, and efficiency of the system, while implementation involved integrating all modules into a fully functional web application.

6.1 SYSTEM TESTING

System testing is the process of evaluating the application to identify errors, verify functionality, and ensure that all modules work properly together. The ETRACKER system was tested under different conditions to ensure reliability and smooth performance.

Types of Testing Performed

1. UNIT TESTING

Each module of the application, such as login, registration, expense management, and dashboard functionalities, was tested individually to verify correct operation.

2. INTEGRATION TESTING

Integration testing was conducted to ensure proper communication between the frontend, backend, and database components of the system.

3. FUNCTIONAL TESTING

Functional testing verified whether the application performed all required operations such as adding expenses, updating records, generating reports, and displaying charts correctly.

4. USER INTERFACE TESTING

The user interface was tested to ensure proper navigation, responsiveness, and ease of use across different pages of the application.

5. SECURITY TESTING

Authentication and session management features were tested to ensure secure access to user accounts and protection of financial data.

6.2 SYSTEM IMPLEMENTATION

System implementation refers to the process of developing, integrating, and deploying the application using the selected technologies and tools.

The ETRACKER system was implemented using the Flask framework for backend development. The frontend was developed using HTML, CSS, and JavaScript to create an interactive and responsive user interface. SQLite was used as the database for storing user details and financial transaction records.

Python libraries such as NumPy, Pandas, and Matplotlib were integrated into the system for performing financial calculations, data analysis, and graphical visualization of expense reports.

The implementation process included:

- Designing user interfaces and web pages
- Creating database tables and relationships
- Developing backend routes and functionalities using Flask
- Integrating login authentication and session handling
- Implementing expense management features
- Generating graphical reports and summaries
- Testing and debugging the application

The application was successfully deployed and tested in a web browser environment. The system operates efficiently and provides users with an effective platform for managing personal finances and household budgets.

7. SYSTEM MAINTENANCE

System maintenance refers to the process of updating, improving, and managing a software system after its development and deployment. It ensures that the application continues to function efficiently, securely, and reliably over time. Maintenance is an important phase in the software development life cycle because it helps identify and correct errors, improve performance, and adapt the system to new requirements or technologies.

In our system, system maintenance is performed to ensure smooth operation of the application and proper management of user financial data. Regular maintenance activities help improve the performance, security, and usability of the system.

TYPES OF MAINTENANCE

1. CORRECTIVE MAINTENANCE

Corrective maintenance involves identifying and fixing errors or bugs that may occur during the operation of the application. This ensures that all modules function correctly without affecting user experience.

2. ADAPTIVE MAINTENANCE

Adaptive maintenance is performed to modify the system according to changes in technology, software updates, or user requirements. It helps the application remain compatible with modern platforms and browsers.

3. PERFECTIVE MAINTENANCE

Perfective maintenance focuses on improving system performance, interface design, and additional functionalities based on user feedback and future enhancements.

4. PREVENTIVE MAINTENANCE

Preventive maintenance is carried out to improve system reliability and avoid future problems by updating software components, optimizing database performance, and improving security mechanisms.

Maintenance Activities in ETRACKER

- Regular database backup and data management
- Updating system security and authentication features
- Fixing software bugs and performance issues
- Improving graphical reports and dashboard functionalities
- Updating libraries and frameworks such as Flask, Pandas, and Matplotlib
- Enhancing user interface and responsiveness

Proper system maintenance ensures that the ETRACKER application remains secure, efficient, and capable of meeting future user requirements effectively.

8. FUTURE ENHANCEMENTS

Future enhancement refers to the additional features and improvements that can be implemented in the system to increase its functionality, efficiency, and user experience. Although the **ETRACKER – Home Based Budget Tracking Mechanism** currently provides efficient expense tracking and financial management features, several advanced functionalities can be added in the future to improve the overall performance and usability of the system.

In future versions, a mobile application can be developed for Android and iOS platforms to provide users with easier accessibility and convenience. Cloud database integration can also be implemented to improve data backup, scalability, and remote access to financial records. Advanced technologies such as Artificial Intelligence and Machine Learning can be integrated to provide smart expense prediction and personalized budgeting suggestions.

The system can further be enhanced by introducing multi-user family budget management features, allowing multiple family members to manage shared expenses and savings together. Integration with online payment systems and banking services can help automate transaction tracking and improve financial monitoring.

Additional improvements such as advanced data analytics, notification and reminder systems, report generation in PDF or Excel format, and enhanced security features like two-factor authentication can also be implemented. Voice assistant support may also be added to improve accessibility and user interaction.

These future enhancements will help make the ETRACKER system more intelligent, scalable, secure, and efficient for modern financial management needs.

9. CONCLUSION

The **ETRACKER – Home Based Budget Tracking Mechanism** was successfully developed as a web-based application for managing personal financial activities efficiently. The system provides a simple, secure, and user-friendly platform that enables users to record, organize, and monitor their income and expenses effectively.

The application was implemented using the Flask framework along with technologies such as HTML, CSS, JavaScript, and SQLite. Python libraries including NumPy, Pandas, and Matplotlib were used for financial calculations, data analysis, and graphical visualization.

The system successfully achieved its objectives by providing secure user authentication, expense categorization, real-time financial tracking, and graphical analysis of spending patterns. The application reduces manual effort, minimizes calculation errors, and helps users maintain better financial discipline and budgeting practices.

Overall, the ETRACKER system serves as an effective solution for home-based budget management and provides a strong foundation for future enhancements and advanced financial management features.

10. BIBLIOGRAPHY

1. For Python

<https://www.python.org/>

<https://docs.python.org/3/>

<https://www.w3schools.com/python/>

1. Mark Lutz, *Learning Python*, O'Reilly Media, 2013.
2. Eric Matthes, *Python Crash Course*, No Starch Press, 2019.

2. For Flask

<https://flask.palletsprojects.com/>

<https://www.tutorialspoint.com/flask/>

<https://www.geeksforgeeks.org/flask-tutorial/>

1. Miguel Grinberg, *Flask Web Development*, O'Reilly Media, 2018.
2. Armin Ronacher, *Flask Framework Documentation*, Pallets Projects.

3. For HTML, CSS and JavaScript

<https://www.w3schools.com/>

<https://developer.mozilla.org/>

<https://www.javascript.com/>

1. Jon Duckett, *HTML and CSS: Design and Build Websites*, Wiley Publications, 2014.
2. Jon Duckett, *JavaScript and JQuery: Interactive Front-End Web Development*, Wiley Publications, 2014.

4. For SQLite

<https://www.sqlite.org/>

<https://www.sqlitetutorial.net/>

<https://www.tutorialspoint.com/sqlite/>

1. Grant Allen, *The Definitive Guide to SQLite*, Apress Publications, 2010.
2. Jay A. Kreibich, *Using SQLite*, O'Reilly Media, 2010.

5. For NumPy

<https://numpy.org/>

<https://numpy.org/doc/>

<https://www.geeksforgeeks.org/numpy-tutorial/>

1. Travis Oliphant, *Guide to NumPy*, USA: Trelgol Publishing, 2006.

2. Ivan Idris, *NumPy Beginner's Guide*, Packt Publishing, 2015.

6. For Pandas

<https://pandas.pydata.org/>

<https://pandas.pydata.org/docs/>

<https://www.geeksforgeeks.org/pandas-tutorial/>

1. Wes McKinney, *Python for Data Analysis*, O'Reilly Media, 2017.
2. Stefanie Molin, *Hands-On Data Analysis with Pandas*, Packt Publishing, 2021.

7. For Matplotlib

<https://matplotlib.org/>

<https://matplotlib.org/stable/tutorials/>

<https://www.geeksforgeeks.org/matplotlib-tutorial/>

1. John D. Hunter, *Matplotlib Documentation*, Matplotlib Development Team.
2. Sandro Tosi, *Matplotlib for Python Developers*, Packt Publishing, 2009.

11. APPENDIX

11.1 SYSTEM CODING

```
from flask import Flask, render_template, request, redirect
from flask import session, Response, send_file
import sqlite3
import bcrypt
import os
import io
import csv
import re
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
app = Flask(__name__)
app.secret_key = "secret123"
DB_FILE = "database.db"
CSV_FILE = "contact-details.csv"

# ----- DATABASE INITIALIZATION -----

def init_db():
    conn = sqlite3.connect(DB_FILE)
    cursor = conn.cursor()
    cursor.execute("""
        CREATE TABLE IF NOT EXISTS users(
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            fullname TEXT NOT NULL,
            email TEXT UNIQUE NOT NULL,
            password BLOB NOT NULL,
            budget REAL DEFAULT 0
        )
    """)
    cursor.execute("""
        CREATE TABLE IF NOT EXISTS expenses(
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            user_email TEXT,
            title TEXT,
            amount REAL,
            category TEXT,
            date TEXT
        )
    """)
    cursor.execute("""
        CREATE TABLE IF NOT EXISTS income(
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            user_email TEXT,
            source TEXT,
```

```

        amount REAL,
        date TEXT
    )
    """
cursor.execute("""
    CREATE TABLE IF NOT EXISTS categories(
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        category_name TEXT UNIQUE NOT NULL
    )
    """)
default_categories = [
    "Food",
    "Travel",
    "Shopping",
    "Bills",
    "Entertainment"
]
for category in default_categories:
    try:
        cursor.execute(
            "INSERT INTO categories(category_name) VALUES(?)",
            (category,)
        )
    except:
        pass
conn.commit()
conn.close()

# ----- HOME PAGE -----

@app.route("/")
def home():

    return render_template("home.html")

# ----- REGISTER -----

@app.route("/register")
def register():
    return render_template("register.html")

@app.route("/register-user", methods=["POST"])
def register_user():

    fullname = request.form.get("fullname")
    email = request.form.get("email")
    password = request.form.get("password")
    if not fullname or not email or not password:

        return render_template(

```

```

        "register.html",
        error="All fields required"
    )

    if not re.match(r"^[^@]+@[^@]+\.[^@]+$", email):
        return render_template(
            "register.html",
            error="Invalid email format"
        )
    hashed_password = bcrypt.hashpw(
        password.encode("utf-8"),
        bcrypt.gensalt()
    )
    try:
        conn = sqlite3.connect(DB_FILE)
        cursor = conn.cursor()
        cursor.execute("""
            INSERT INTO users(fullname,email,password)
            VALUES(?,?,?)
            """, (
                fullname,
                email,
                hashed_password
            ))
        conn.commit()
        conn.close()
        return render_template(
            "login.html",
            success="Registration successful"
        )
    except sqlite3.IntegrityError:
        return render_template(
            "register.html",
            error="Email already exists"
        )

# ----- LOGIN -----

@app.route("/login")
def login():

    return render_template("login.html")

@app.route("/login-user", methods=["POST"])
def login_user():
    email = request.form.get("email")
    password = request.form.get("password")
    conn = sqlite3.connect(DB_FILE)
    cursor = conn.cursor()

```

```

cursor.execute("""
    SELECT fullname,email,password
    FROM users
    WHERE email=?
""", (email,))

user = cursor.fetchone()
conn.close()
if not user:
    return render_template(
        "login.html",
        error="Email not found"
    )
stored_password = user[2]
if isinstance(stored_password, str):
    stored_password = stored_password.encode("utf-8")
if not bcrypt.checkpw(
    password.encode("utf-8"),
    stored_password
):
    return render_template(
        "login.html",
        error="Wrong password"
    )
session["user"] = user[0]
session["email"] = user[1]
return redirect("/dashboard")

# ----- DASHBOARD -----

@app.route("/dashboard")
def dashboard():
    if "user" not in session:
        return redirect("/login")
    conn = sqlite3.connect(DB_FILE)
    conn.row_factory = sqlite3.Row
    cursor = conn.cursor()
    cursor.execute("""
        SELECT * FROM expenses
        WHERE user_email = ?
""", (session["email"],))
    expenses = cursor.fetchall()
    cursor.execute("""
        SELECT SUM(amount)
        FROM expenses
        WHERE user_email = ?
""", (session["email"],))
    total_expense = cursor.fetchone()[0] or 0
    cursor.execute("""
        SELECT SUM(amount)

```

```

        FROM income
        WHERE user_email = ?
    """ , (session["email"],))
    total_income = cursor.fetchone()[0] or 0
    balance = total_income - total_expense

# ----- FETCH CATEGORIES -----

    cursor.execute("""
        SELECT *
        FROM categories
        ORDER BY category_name ASC
    """)
    categories = cursor.fetchall()
    conn.close()
    return render_template(
        "dashboard.html",
        expenses=expenses,
        total=total_expense,
        income=total_income,
        balance=balance,
        categories=categories
    )
# ----- ADD EXPENSE -----

@app.route("/add-expense", methods=["GET", "POST"])
def add_expense():
    if "user" not in session:
        return redirect("/login")
    conn = sqlite3.connect(DB_FILE)
    conn.row_factory = sqlite3.Row
    cursor = conn.cursor()
    cursor.execute("""
        SELECT *
        FROM categories
        ORDER BY category_name ASC
    """)
    categories = cursor.fetchall()
    if request.method == "POST":
        title = request.form.get("title")
        amount = request.form.get("amount")
        category = request.form.get("category")
        date = request.form.get("date")
        if not title or not amount or not category:
            return render_template(
                "add_expense.html",
                error="All fields required",
                categories=categories
            )

```



```

cursor.execute("""
    INSERT INTO expenses
    (user_email,title,amount,category,date)
    VALUES(?,?,?,?,?)
""", (
    session["email"],
    title,
    amount,
    category,
    date
))
conn.commit()
conn.close()
return redirect("/dashboard")
return render_template(
    "add_expense.html",
    categories=categories
)

```

----- ANALYTICS -----

```

@app.route("/chart/pie")
def chart_pie():
    if "user" not in session:
        return redirect("/login")
    conn = sqlite3.connect(DB_FILE)
    cursor = conn.cursor()
    cursor.execute("""
        SELECT category,SUM(amount)
        FROM expenses
        WHERE user_email=?
        GROUP BY category
    """, (session["email"],))
    data = cursor.fetchall()
    conn.close()
    categories = [row[0] for row in data]
    amounts = [row[1] for row in data]
    plt.figure(figsize=(6,6))
    plt.pie(
        amounts,
        labels=categories,
        autopct='%1.1f%%'
    )
    img = io.BytesIO()
    plt.savefig(
        img,
        format='png',
        bbox_inches='tight'
    )

```

```

img.seek(0)
plt.close()
return send_file(
    img,
    mimetype='image/png'
)

# ----- LOGOUT -----

@app.route("/logout")
def logout():
    session.clear()
    return redirect("/")

# ----- RUN APPLICATION -----

if __name__ == "__main__":
    init_db()
    app.run(
        host="0.0.0.0",
        port=int(os.environ.get("PORT", 5000))
    )

```

11.2 SCREENSHOTS

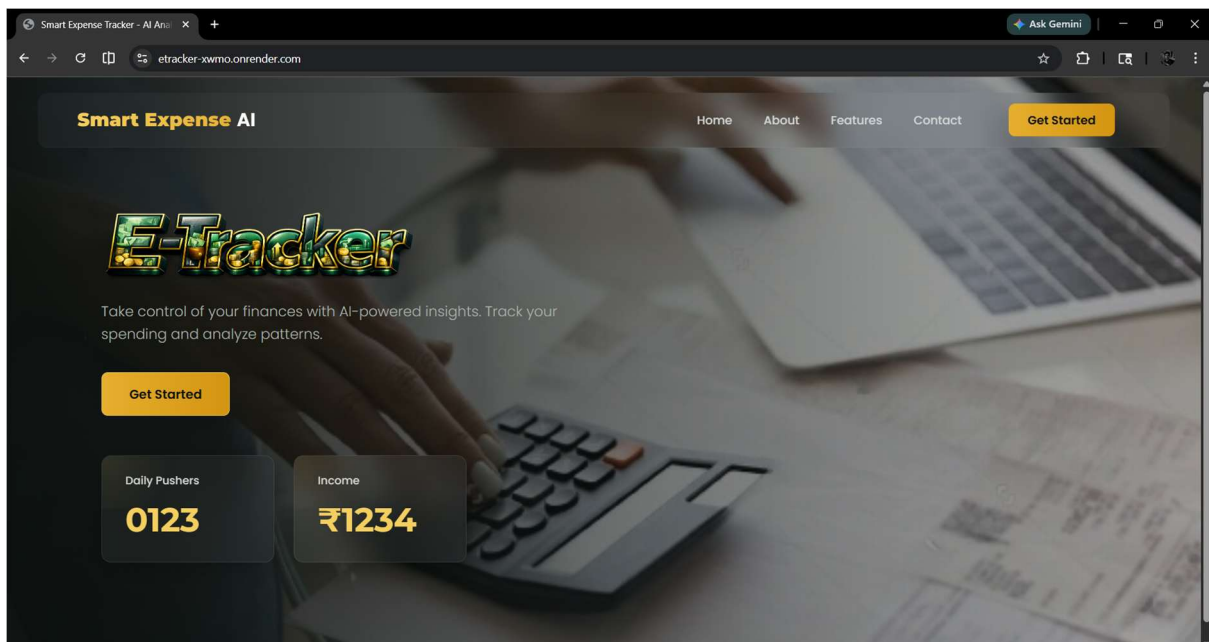


fig 11.2.1. Home

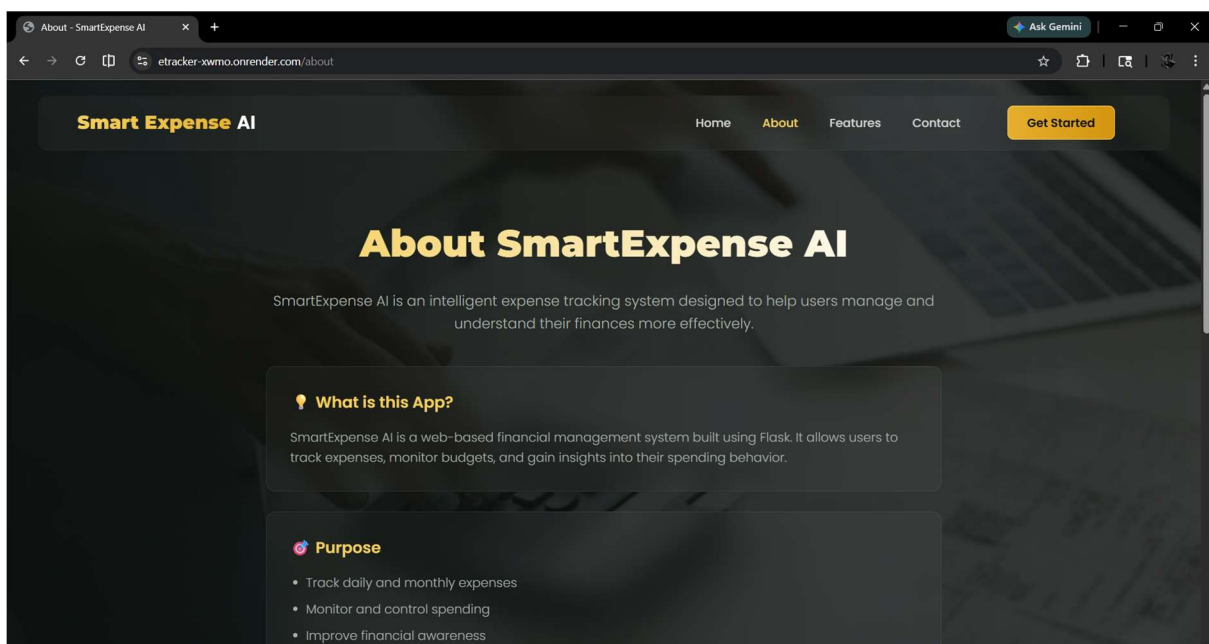


fig 11.2.2. About

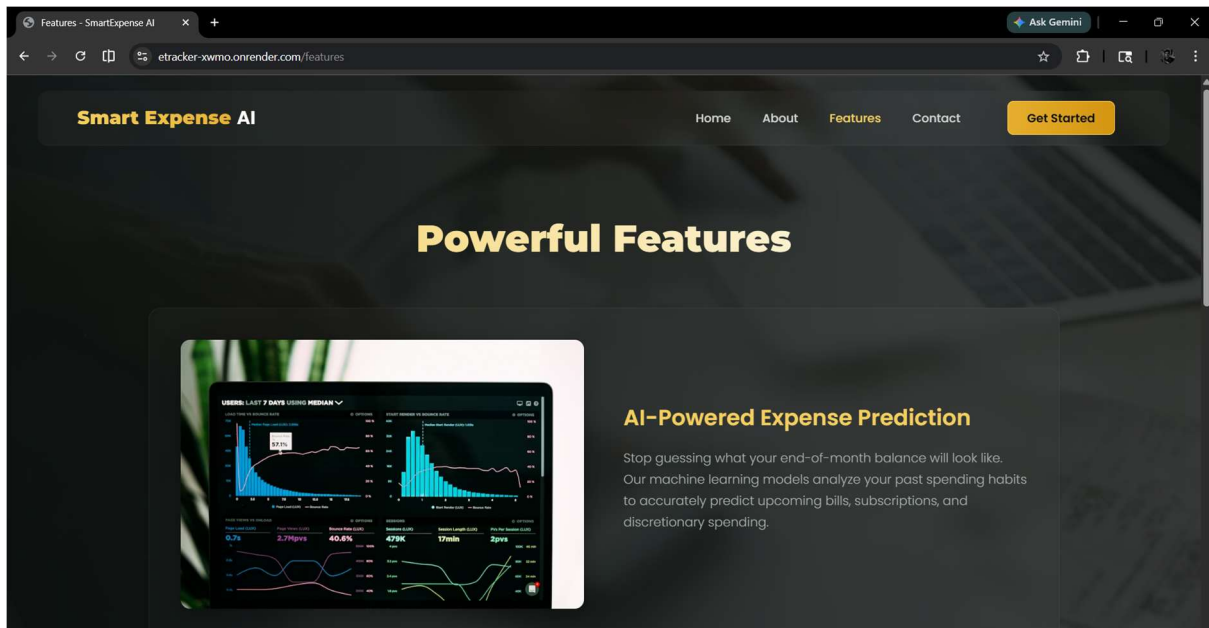


fig 11.2.3. Features

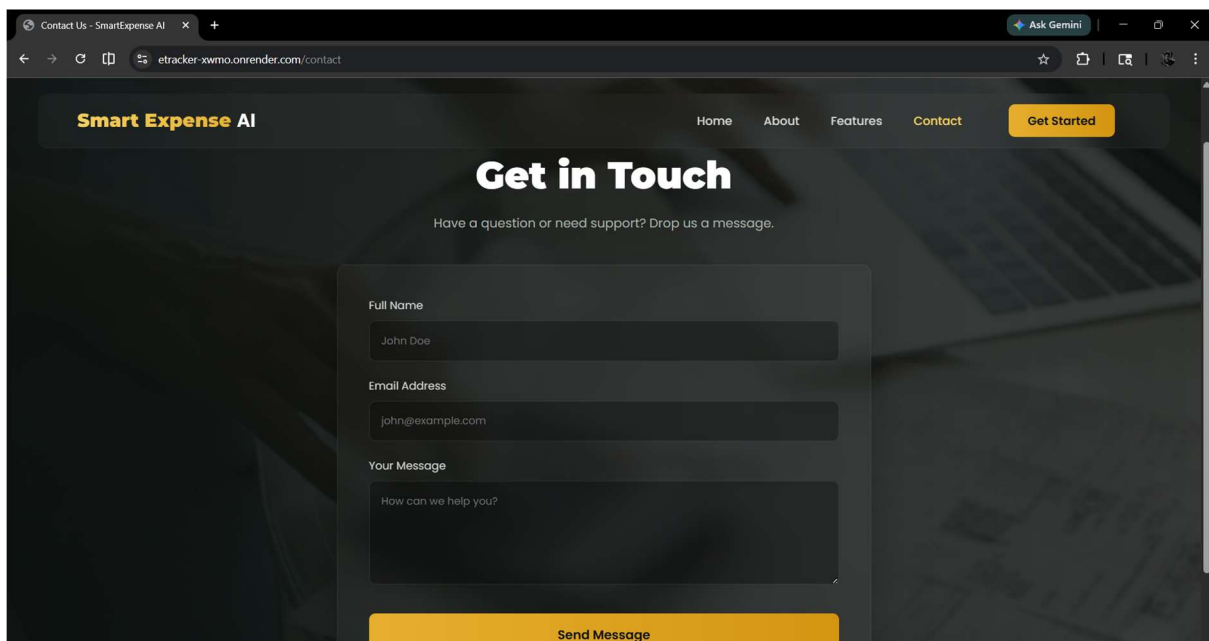


fig 11.2.4. Contact

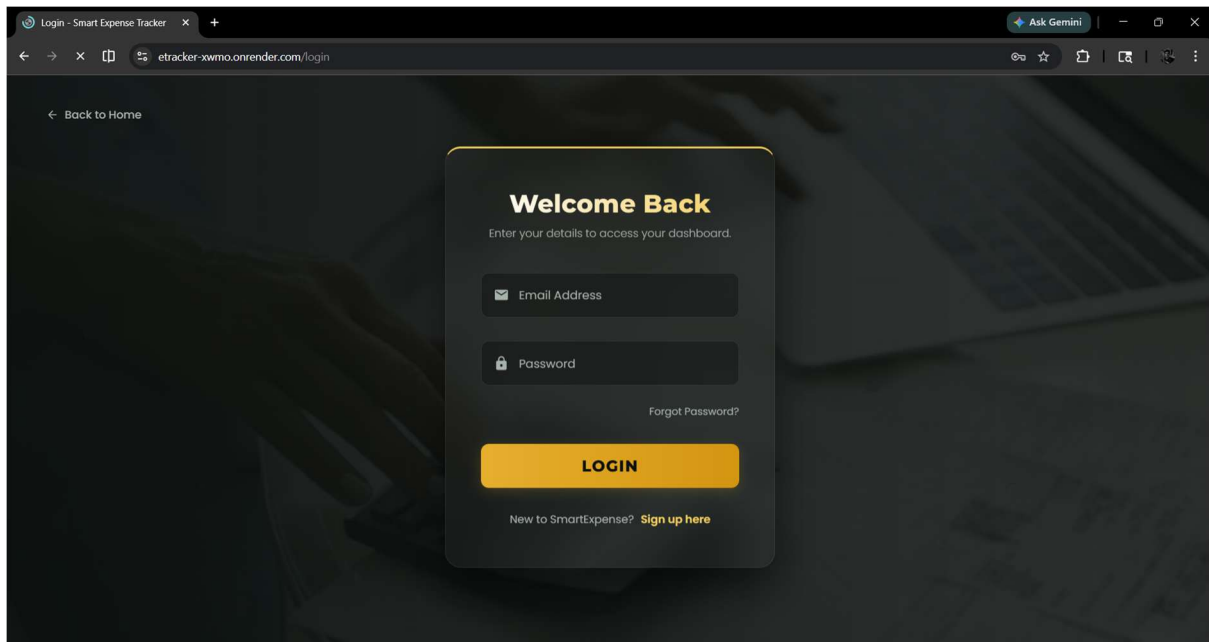


fig 11.2.5. Login

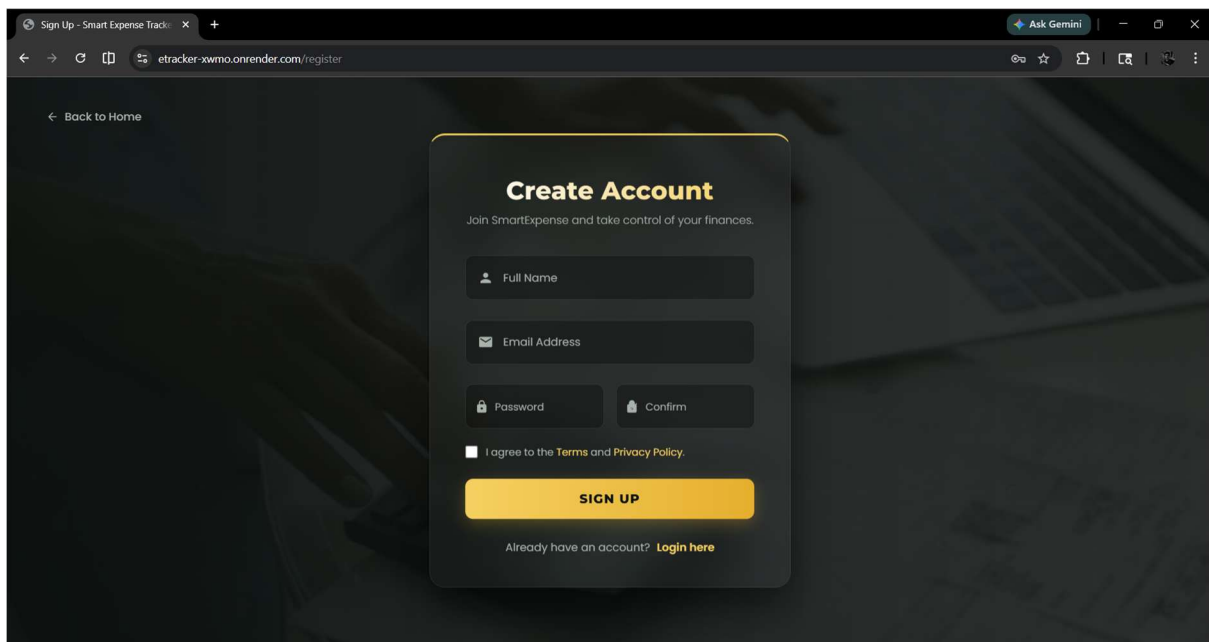


fig 11.2.6. Signup

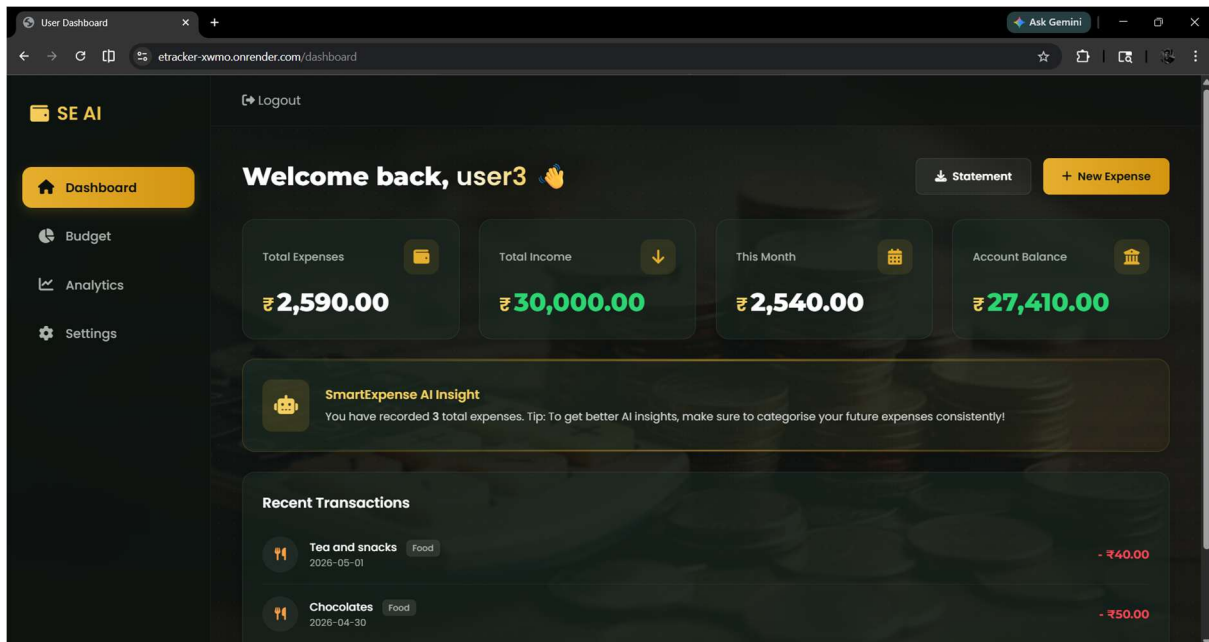


fig 11.2.7. User Dashboard

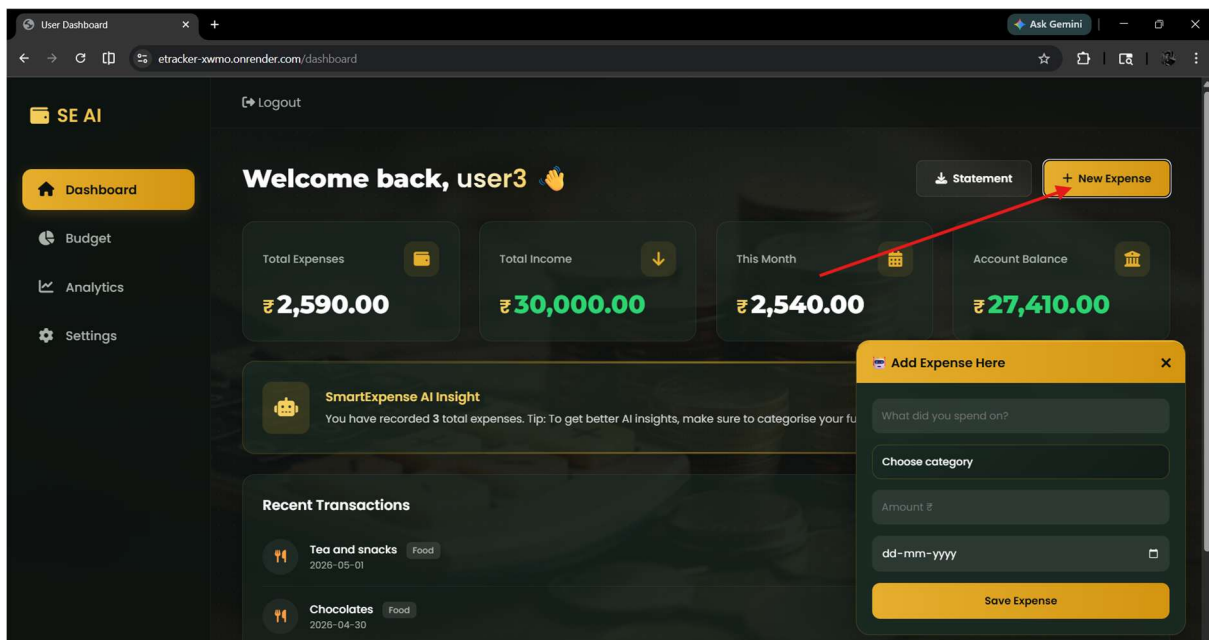


fig 11.2.8. Add Expense bot

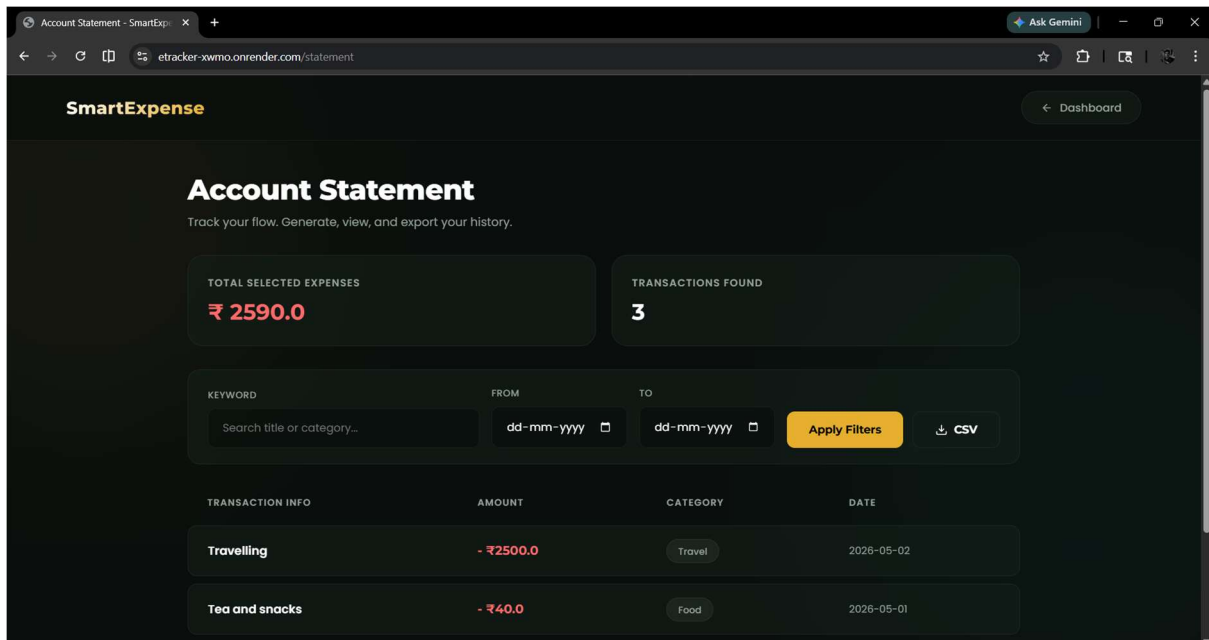


fig 11.2.9. Statement

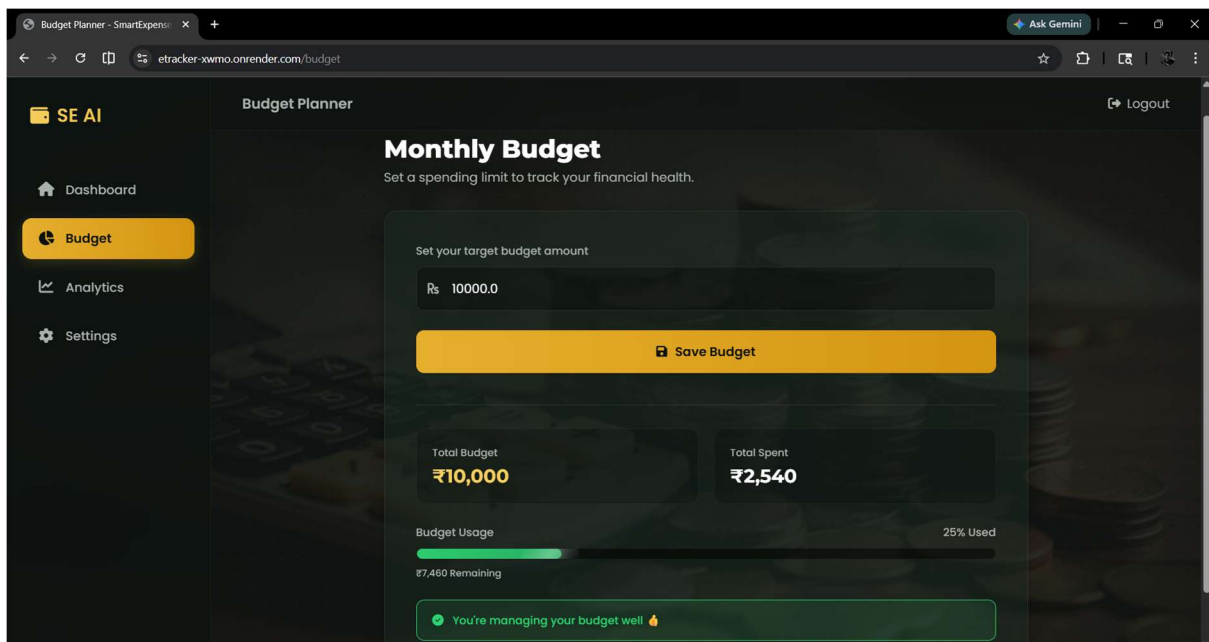


fig 11.2.10. Budget Planner

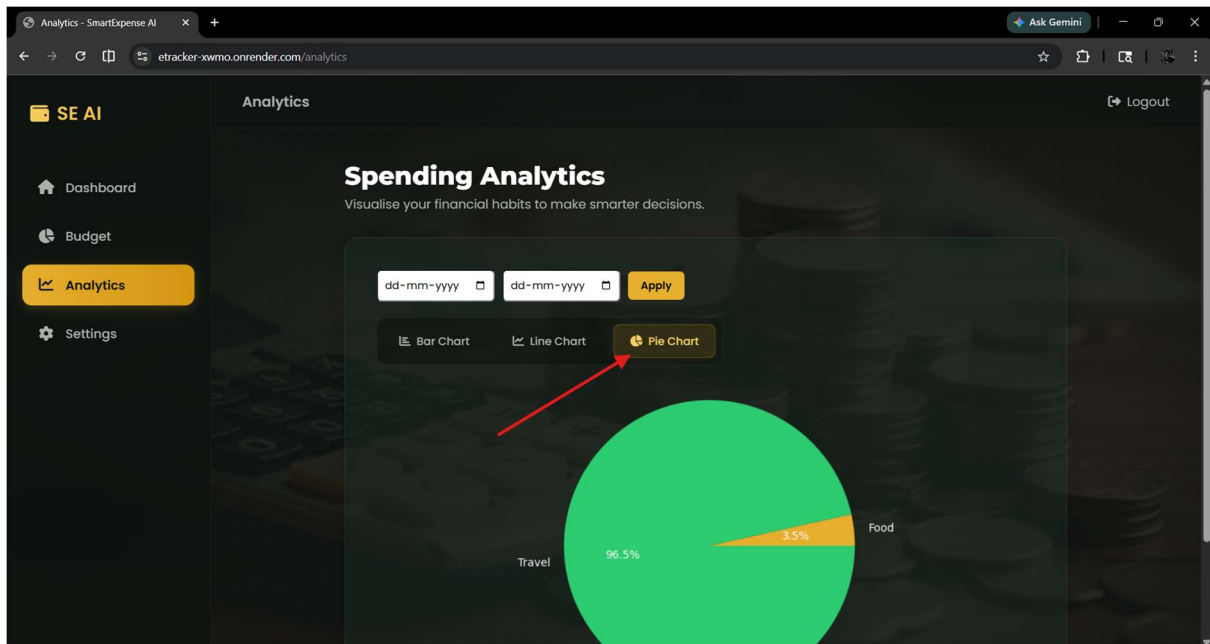


fig 11.2.11. Analytics – Pie chart

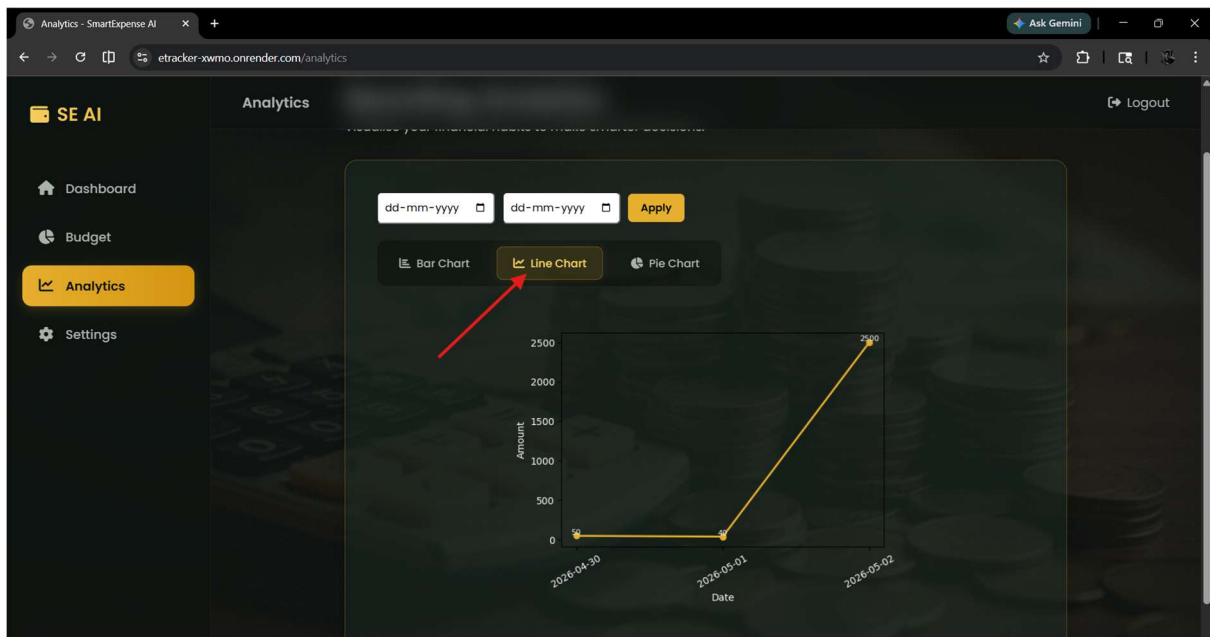


fig 11.2.12. Analytics – Line chart

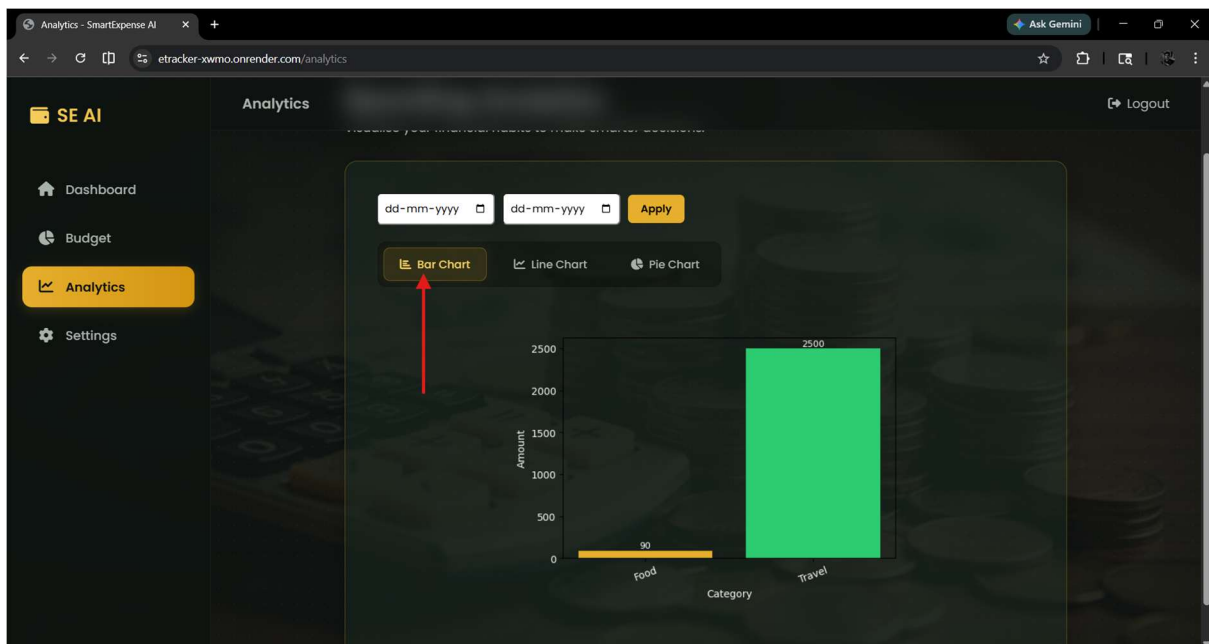


fig 11.2.13. Analytics – Bar chart

The screenshot shows the 'Settings' page of the 'SmartExpense AI' application. The left sidebar contains navigation links: Dashboard, Budget, Analytics, and Settings (highlighted). The main content area displays the 'Account Settings' section. The 'Monthly Budget' tab is selected, and the 'Update Monthly Limit' dialog is open. The dialog shows the 'BUDGET AMOUNT (INR)' set to 10000.0 and a 'Save Budget' button. A red arrow points to the 'Monthly Budget' tab.

Tab	Value
Monthly Budget	10000.0

fig 11.2.14. Settings – Budget fixing

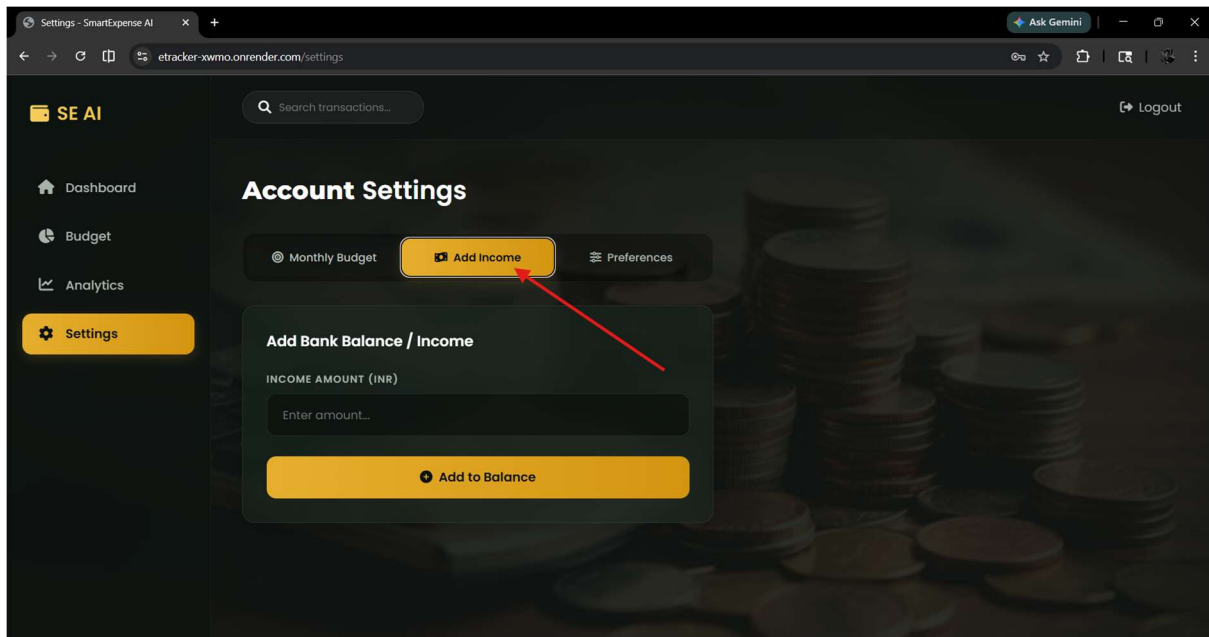


fig 11.2.15. Settings – Adding Income

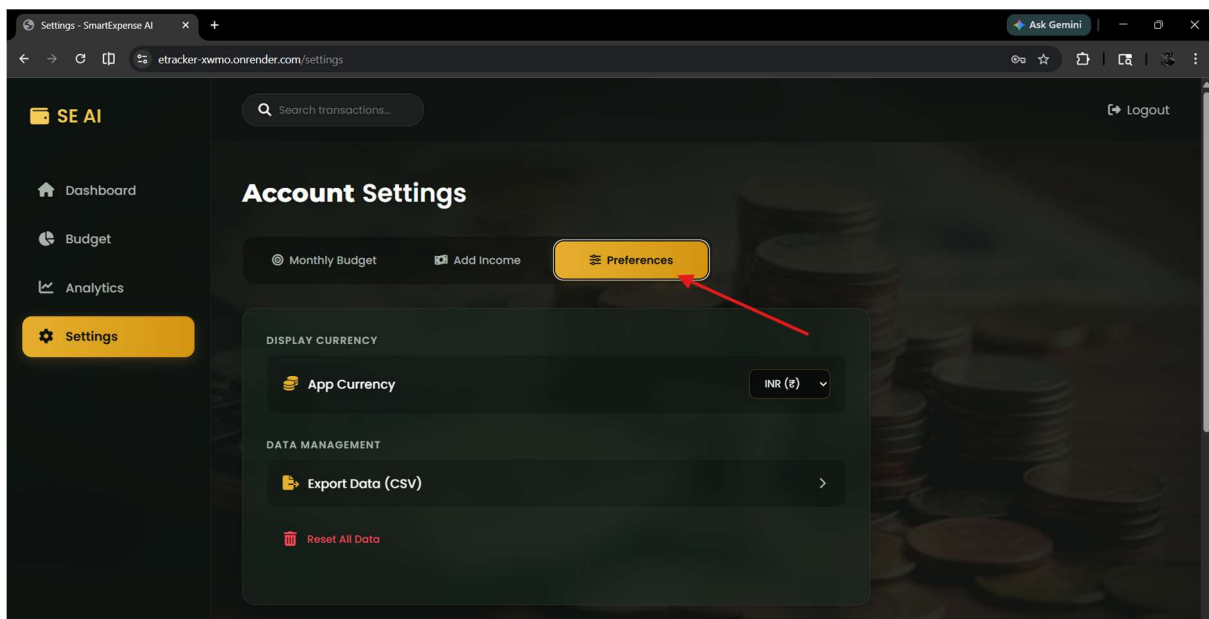


fig 11.2.16. Settings – preferences

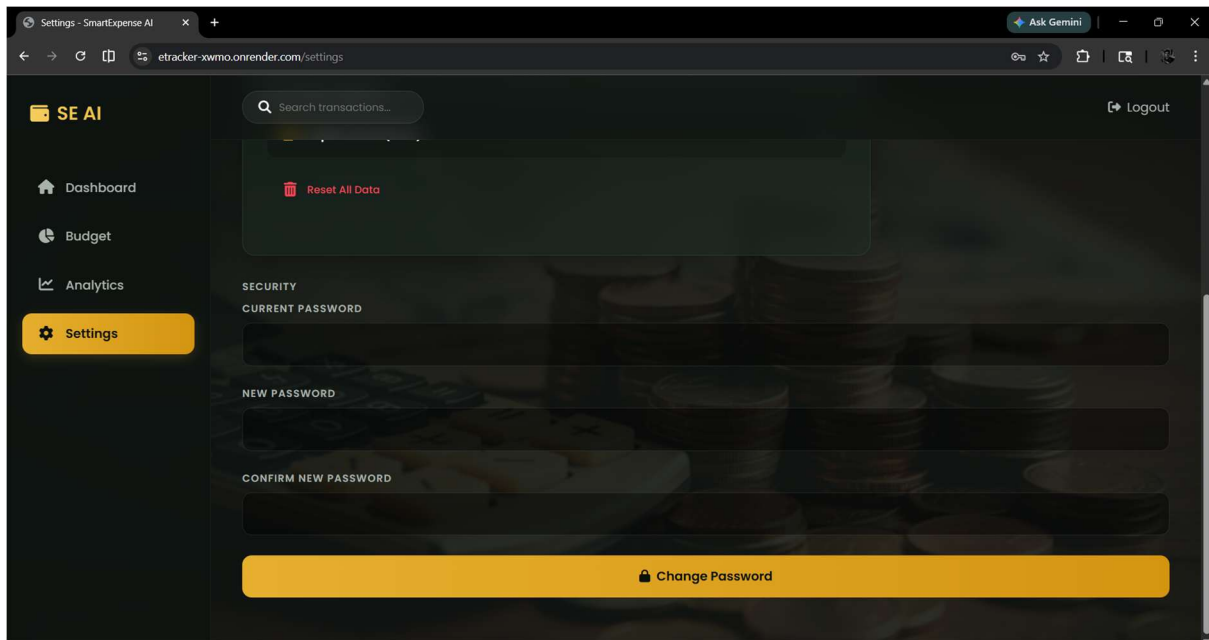


fig 11.2.17. Settings – security feature

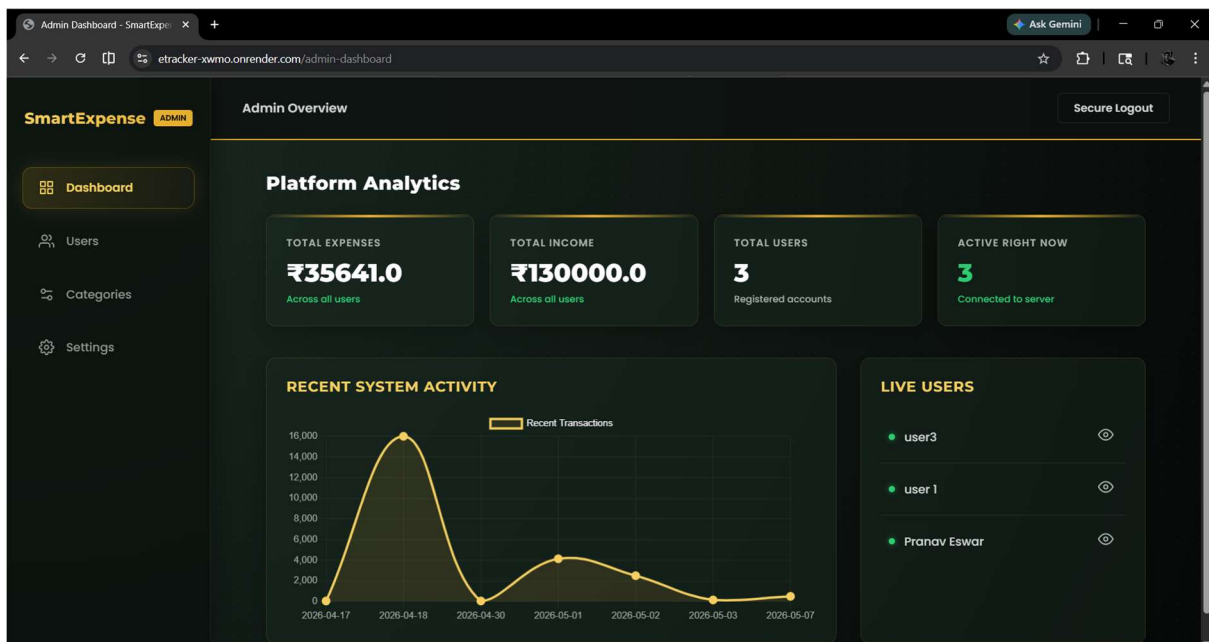


fig 11.2.18. Admin dashboard

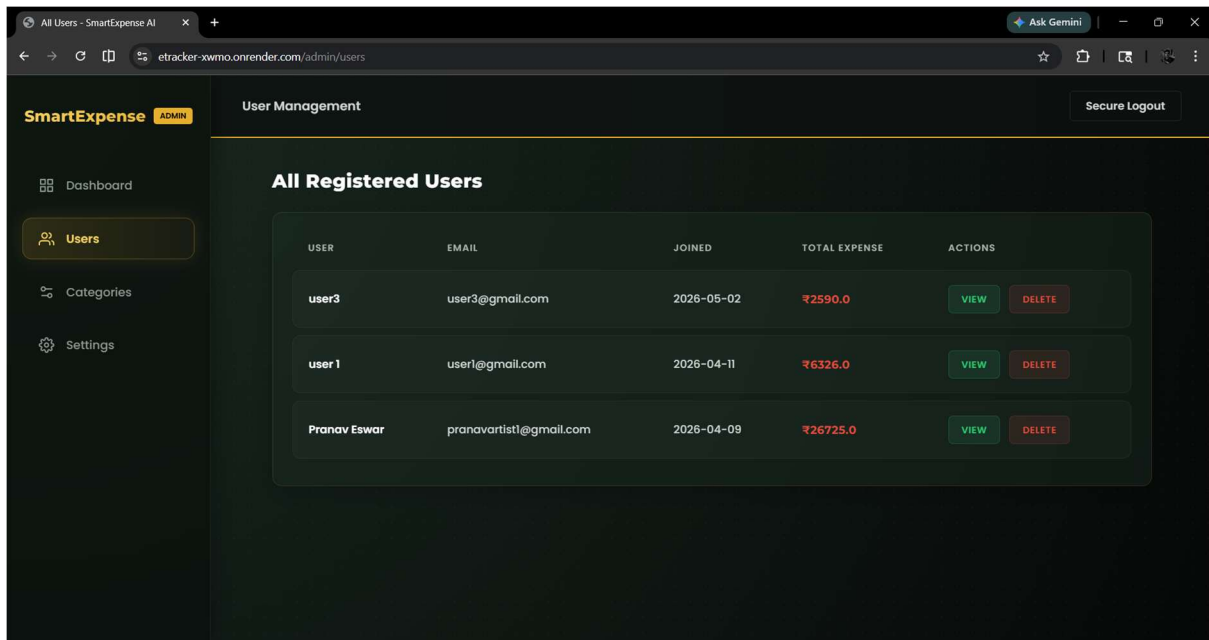


fig 11.2.19. Admin – Handles Users

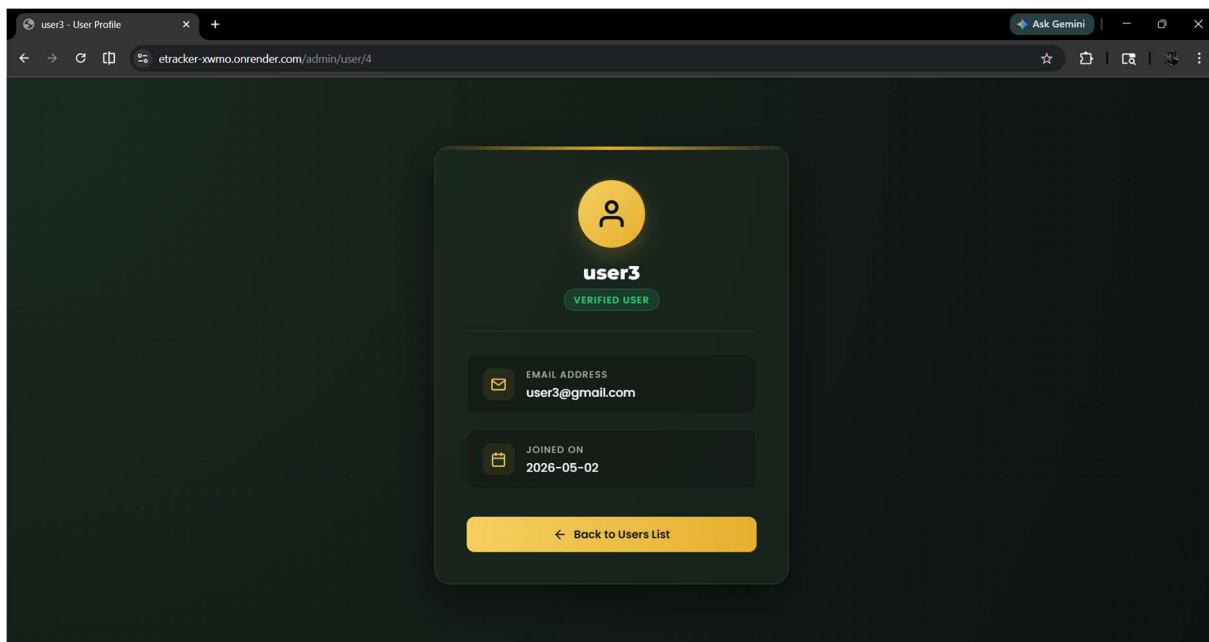


fig 11.2.20. Admin – Each user profile

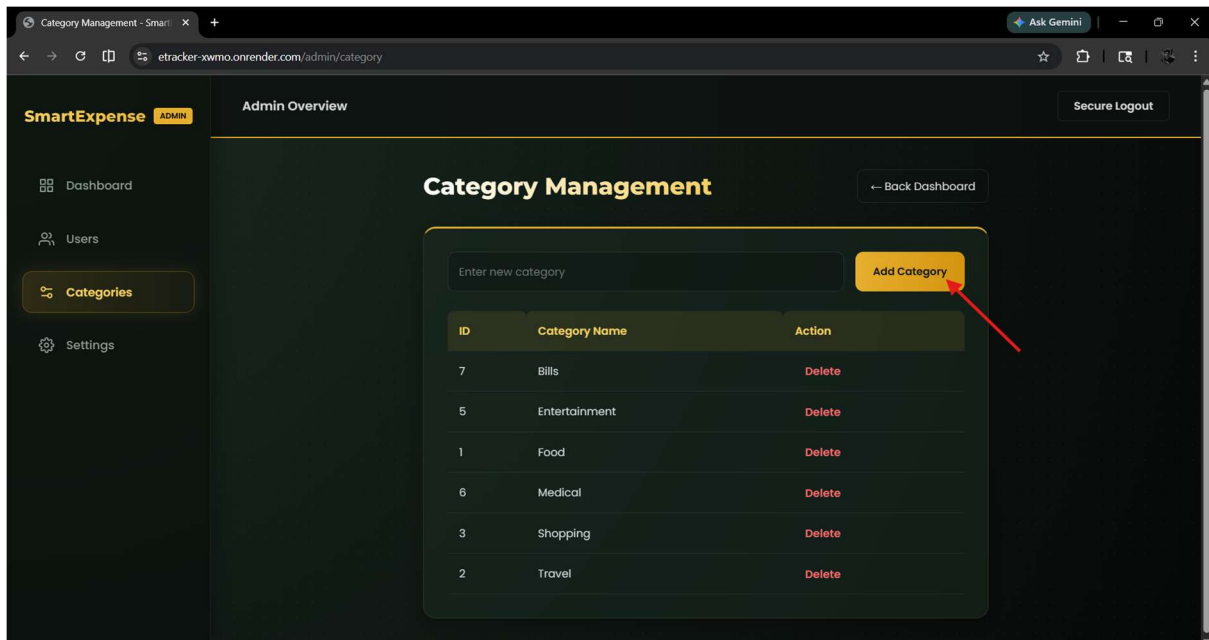


fig 11.2.21. Admin – Add/Delete categories

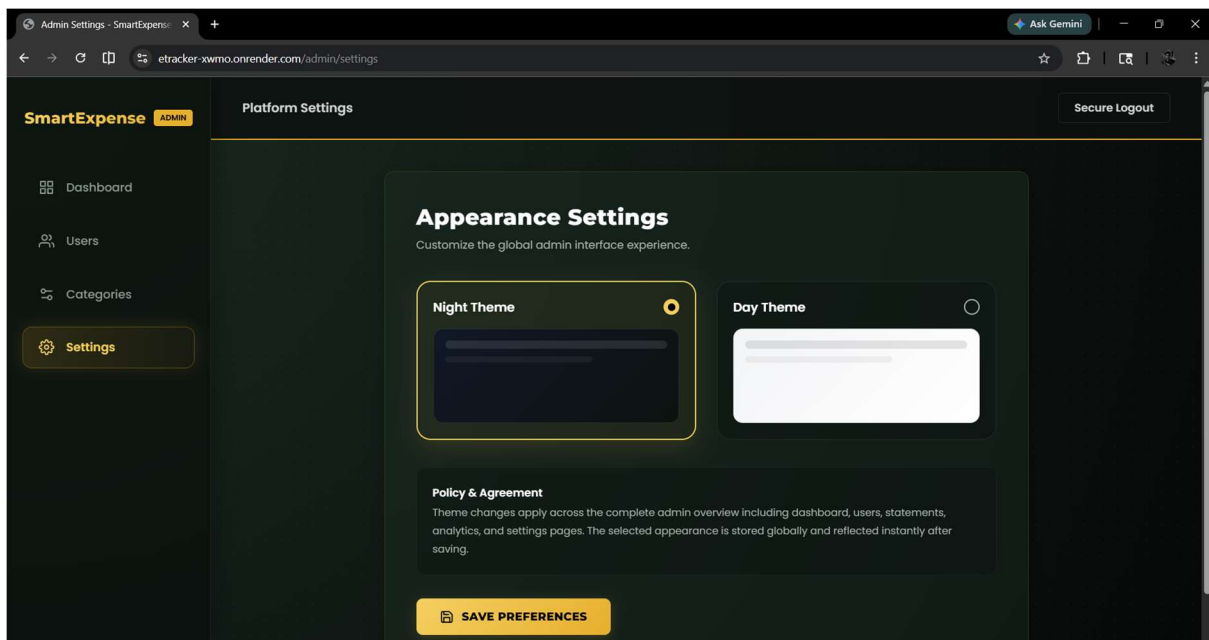


fig 11.2.22. Admin – Settings